

Brazilian E-Commerce Public Dataset

2. Data Integration Report (Updated)

Prepared by

Ragavendra. R

AI / ML Engineer

1. Overview of Changes

- The data_integration.ipynb notebook was updated with one structural addition: a formal relationship mapping layer documenting all 9 join relationships across order_level and item_level.
- The relationship map is exported as relationship_mapping.csv (9 rows x 7 cols) in the reports/ folder -- now the authoritative data lineage reference for this project.
- The two output tables (order_level and item_level) are unchanged in shape and content. This update is documentation-only, not a logic change.
- Updated notebook also clarifies item_level column progression: 21 columns after the 4-table join chain, rising to 22 after the derived item_revenue column is added.

2. Input Tables

- All 9 sampled tables loaded successfully from data/sampled data/ as confirmed by notebook output.

Table	Rows	Cols	Role in Integration
orders_s	10,000	8	Central fact -- base for order_level
customers_s	10,000	5	Customer dimension -- joined on customer_id
order_items_s	11,383	7	Line-item fact -- base for item_level
payments_s	10,476	5	Payment dimension -- pre-aggregated before join
reviews_s	9,959	7	Review dimension -- pre-aggregated before join
products_s	6,747	9	Product dimension -- joined on product_id
sellers_s	1,654	4	Seller dimension -- joined on seller_id
category_tr_s	71	2	Translation lookup -- joined on product_category_name
geo_clean	6,486	5	Geo dimension -- joined on zip_code_prefix

- order_items (11,383) outnumber orders (10,000) -- 1,383 orders had multiple items.
- payments (10,476) exceed orders by 476 -- some orders had split payments or multiple installment rows.
- reviews (9,959) fall short of orders by 41 -- those orders completed but received no customer feedback.

3. Pre-Aggregation Before Join

- Payments and reviews were aggregated at order_id grain before joining to prevent row fan-out in order_level.

3.1 Payments Aggregation (pay_agg)

Function	Source Column	Output Column	Logic
SUM	payment_value	total_payment_value	Total amount paid per order
MAX	payment_installments	max_installments	Highest installment count across payment rows
NUNIQUE	payment_type	n_payments_methods	Number of distinct payment methods used
MODE	payment_type	main_payment_type	Most frequent payment method for the order

3.2 Reviews Aggregation (rev_agg)

Function	Source Column	Output Column	Logic
MEAN	review_score	avg_review_score	Average score if multiple reviews exist per order
NUNIQUE	review_id	n_reviews	Count of distinct reviews per order

4. order_level Table Build

- Grain: one row per order. Final shape confirmed: 10,000 rows x 23 columns.
- Built by chaining four sequential LEFT JOINS with orders_s as the base table.

Step	Joined Table	Join Key	Columns Added	Shape After
Base	orders_s	--	8 base columns	10,000 x 8
1	customers_s	customer_id	customer_unique_id, zip, city, state (+4)	10,000 x 12
2	pay_agg	order_id	total_payment_value, max_installments, n_methods, main_type (+4)	10,000 x 16
3	rev_agg	order_id	avg_review_score, n_reviews (+2)	10,000 x 18
4	geo_clean	customer_zip_code_prefix	geo_zip, lat, lng, geo_city, geo_state (+5)	10,000 x 23

- Verified: all 10,000 rows survived all four joins -- zero fan-out, zero row loss.
- Geo join uses a non-standard key match: customer_zip_code_prefix (left) to geolocation_zip_code_prefix (right).
- 35 orders (0.35%) had zip codes absent from geo_clean -- these rows carry NaN for lat, lng, geo_city, geo_state.

5. item_level Table Build

- Grain: one row per order line item. Final shape confirmed: 11,383 rows x 22 columns.
- Built by chaining four sequential LEFT JOINS on order_items_s as the base, then one derived column step.

Step	Joined Table	Join Key	Columns Added	Shape After
Base	order_items_s	--	7 base columns	11,383 x 7
1	products_s	product_id	category_name, dimensions, weight (+8)	11,383 x 14
2	category_tr_s	product_category_name	product_category_name_english (+1)	11,383 x 15
3	sellers_s	seller_id	seller_zip, seller_city, seller_state (+3)	11,383 x 18
4	orders_s	order_id	order_purchase_timestamp, order_status (+2)	11,383 x 20
5	Derived col	--	item_revenue = price + freight_value (+1)	11,383 x 22

- Column count note: first head() in notebook shows 21 columns (after joins, before derived col). Final confirmed shape is 22 columns after item_revenue is added -- consistent with prior output.
- Verified: all 11,383 rows survived the four-table chain with no duplicates or drops.
- Verified: item_revenue = price + freight_value for all rows -- no arithmetic errors.

6. Relationship Mapping -- New Addition

- A formal relationship map was added to the notebook and exported to reports/relationship_mapping.csv.
- It documents all 9 join relationships with parent/child keys, cardinality, join type, and a plain-English business note.
- This file is now the authoritative data lineage reference for any downstream consumer of order_level or item_level.

6.1 order_level Joins (5 relationships)

Parent Table	Parent Key	Child Key	Cardinality	Join	Business Note
customers_s	customer_id	customer_id	1:M	LEFT	One customer can have multiple orders
orders_s	order_id	order_id	1:1	LEFT	Each order appears exactly once in order_level
pay_agg	order_id	order_id	1:1	LEFT	Payment summary pre-aggregated and merged 1:1
rev_agg	order_id	order_id	1:1	LEFT	Review summary pre-aggregated and merged 1:1
geo_clean	geolocation_zip_code_prefix	customer_zip_code_prefix	1:M	LEFT	Customer location added by zip-code lookup

6.2 item_level Joins (4 relationships)

Parent Table	Parent Key	Child Key	Cardinality	Join	Business Note
products_s	product_id	product_id	1:M	LEFT	One product can appear in many order items
category_tr_s	product_category_name	product_category_name	1:M	LEFT	Category translation attached to each product row
sellers_s	seller_id	seller_id	1:M	LEFT	One seller can sell across many items
orders_s	order_id	order_id	1:M	LEFT	One order can contain multiple line items

- All 9 joins use LEFT type -- preserves every base-table row even when no match exists on the right side.
- All 1:M relationships are resolved before joining by pre-aggregating payments and reviews to order_id grain, ensuring no row fan-out in order_level.
- customers -> order_level is documented as 1:M (one customer can technically appear in multiple orders), but each order_level row has a distinct customer_id, so no duplication occurs in practice.
- The cardinality on geo_clean -> order_level is 1:M because multiple customers can share the same zip code prefix, but the join is still safe since each order_level row has one customer zip.

7. Output Summary

Output File	Rows	Columns	Location	Status
order_level.csv	10,000	23	data/sampled data/	Unchanged
item_level.csv	11,383	22	data/sampled data/	Unchanged
relationship_mapping.csv	9	7	reports/	New -- added this version

- order_level and item_level are identical to the prior version -- no data changes, only documentation added.
- relationship_mapping.csv columns: parent_table, parent_key, child_table, child_key, cardinality, join_type, notes.

8. Post-Integration Integrity Summary

Check	Result
order_level row count	10,000 rows -- matches orders_s base -- PASS
item_level row count	11,383 rows -- matches order_items_s base -- PASS
Fan-out prevention	Payments and reviews pre-aggregated before joining -- PASS
No duplicates in either table	Confirmed -- PASS
item_revenue formula	price + freight_value verified for all 11,383 rows -- PASS
Geo NaN rows	35 orders (0.35%) unmatched -- expected, no action needed
Review NaN rows	88 orders (0.88%) unreviewed -- expected, flagged via has_review
Relationship mapping completeness	9 of 9 join relationships documented -- PASS
All join types	LEFT on all 9 relationships -- consistent and intentional